CHATVIA: Real-Time Confidential Chat with Steganography & Biometrics

A Zero-Trust Confidential Communication Platform Featuring AI-Powered Continuous Biometric Gating, Client-Side Randomized Steganography, and Ephemeral Forensic Self-Healing

1. Project Details

Project Title:	Chatvia (Real-Time Confidential Messaging with AI Biometrics & Steganography)
Team Name:	**Veltrox**
Team Number / ID:	**Team 65** (Track: Cybersecurity & Applied AI, HackMITTen 2026)
Live Cloud Application:	https://chatvia-app-k7o7.onrender.com/
GitHub Repository:	https://github.com/Kishan-Kumar-Hv/hmit
Submission Phase:	Phase 4 – Final End-to-End Project Submission (Phases 1 through 3 Journey)

Team Member Name	Role in Team 65 (Veltrox)	Key Technical Responsibilities & Contributions
Kishan Kumar H V	Team Lead & Cloud/DevOps Architect	Production PaaS deployment on Render, Express 5 reverse proxy & static SPA integration, PostgreSQL Supabase SSL pooler engineering, system integration, automated build pipelines, and quality assurance.
Santhosh J R	Full-Stack & Biometrics Core Engineer	FastAPI AI microservice integration with InspireFace SDK, face enrollment & 512-d feature extraction, real-time WebSocket communication hub, active camera surveillance loop, and alert UI notifications.
Suhas B R	Security & Steganography Architect	Client-side Web Crypto API pipeline (AES-256-GCM), friendship secret agreement with PBKDF2/Argon2 salted hashing, non-exportable IndexedDB key persistence, and randomized LSB steganography algorithms.
Dhruva Kumar V M	Frontend Lead & UI/UX Specialist	Modern React 19 SPA architecture, responsive Tailwind styling with dark/light themes, WebRTC camera lifecycle management, attachment uploading handling, comprehensive test execution, and final documentation.

2. Problem Statement

Original Problem Statement Assigned to Team 65:
"Design and engineer a real-time secure communication system that eliminates vulnerabilities in both data transmission and physical message presentation. The application must guarantee that confidential exchanges remain fully protected from over-the-shoulder eavesdropping, unauthorized physical device access, and persistent forensic extraction, while providing zero-knowledge verifiable privacy and high usability."

3. Understanding the Problem

Modern consumer messaging applications (such as Signal, WhatsApp, and Telegram) have made tremendous strides in securing data in transit using End-to-End Encryption (E2EE). However, Team 65 identified critical gaps in the modern cybersecurity threat model that conventional applications leave completely unaddressed:

- **The 'Last-Inch' Presentation Vulnerability:** While ciphertexts cannot be deciphered in transit or on cloud servers, the moment a message is decrypted onto the user's screen, it becomes vulnerable to physical line-of-sight observation (shoulder surfing in public transit, office spaces, or coffee shops) and unauthorized device seizure.
- **Static vs. Continuous Authentication:** Traditional biometric locks (Face ID, Touch ID) authenticate an individual once when opening the app. If the device is left on a desk, passed to a colleague, or grabbed, any bystander can read confidential conversations.
- **Forensic Residue & Media Caching:** Most messaging apps write received attachments to local device flash storage or gallery folders. Even after deletion, file traces remain recoverable from SQLite databases, operating system thumbnails, and write-ahead logs.
- **Traffic Observability & Lack of Plausible Deniability:** Standard encrypted chats openly signal to network monitors (ISPs, network firewalls) that an encrypted exchange is underway. There is no covert carrier to conceal the existence of confidential communication.

4. Phase 1 – Initial Analysis & Ideation

During Phase 1, Team 65 engaged in extensive brainstorming and offline architectural workshops. We analyzed potential threat models and conceptualized our solution on a cardboard sheet and whiteboard to chart the data flow before writing any code:

- **Cardboard Sheet Architecture Conception:** We mapped out a three-tier defense model on our physical cardboard diagram: (1) An Identity Tier using facial biometrics to anchor account ownership, (2) A Steganographic Cryptographic Tier to disguise secrets within ordinary photographic images, and (3) An Active Client Presentation Tier to enforce viewing rules dynamically.
- **Discarded Approaches:** We initially evaluated client-side WebAssembly face detection (e.g. MediaPipe / face-api.js), but discovered severe thermal throttling, slow load times, and vulnerability to browser inspection. We therefore architected a dedicated backend AI microservice running the state-of-the-art InspireFace deep learning SDK with cosine similarity thresholding.
- **Privacy-First Biometric Decision:** We established a fundamental ethical guideline during Phase 1: *Never store raw camera images of users*. Instead, raw photos are processed in volatile memory, converted to 512-dimensional vector embeddings, and encrypted via AES-256-GCM before saving to PostgreSQL.
- **Steganography Strategy:** Rather than sequential LSB insertion (which creates recognizable visual patterns and can be stripped by simple steganalysis), we designed a randomized PRNG-based LSB distribution seeded with a cryptographically secure token.

5. Requirements Analysis

Category	Requirement ID	Specification & Architectural Rationale
User Needs	UN-01 / UN-02	Confidential communications for corporate executives, legal advisors, and privacy advocates needing protection against physical snooping and post-read digital discovery.
Objectives	OBJ-01 OBJ-02	1. Provide continuous facial biometrics while reading secret messages. 2. Disguise encrypted text within lossless PNG carriers with zero visual distortion.
Constraints	CON-01 CON-02	1. Must run seamlessly in modern web browsers without native desktop plugins. 2. Deployable on unified single-domain cloud PaaS (Render + Supabase).
Functional Requirements	FR-01: Biometrics FR-02: Secrets FR-03: Stego FR-04: Real-Time FR-05: Ephemeral	• Mandatory one-face biometric enrollment with explicit consent. • Out-of-band friendship passphrase with PBKDF2 client-side key derivation. • Randomized LSB embedding of AES-GCM ciphertexts into PNG images. • Real-time bilateral WebSocket messaging and typing indicators. • One-time consumption with automated database restoration of original carrier PNG.
Non-Functional Requirements	NFR-01: Security NFR-02: Speed NFR-03: Privacy	• Sub-100ms message delivery; sub-80ms biometric verification response. • Zero plaintext or raw face photos stored in database or transmitted in tokens. • Multi-layer AES-256-GCM encryption across biometrics and messages.

6. Proposed Solution: Chatvia Architecture

Chatvia solves the end-to-end security paradox by synthesizing **Client-Side Cryptography**, **Randomized Digital Steganography**, and **AI-Powered Continuous Biometric Gating** into an integrated zero-trust platform:

1. **Zero-Knowledge Biometric Enrollment:** Users register using camera capture. InspireFace detects the face and extracts a 512-dimensional normalized embedding vector. The backend encrypts this template with AES-256-GCM using an isolated server master key before database insertion. The raw facial photograph is immediately purged from memory.
2. **Mutual Passphrase & Key Derivation:** Communicating parties establish a friendship passphrase. The browser uses the Web Crypto API to derive an AES-256-GCM key via PBKDF2 with a random salt. This key is marked non-exportable and cached locally in browser IndexedDB. The server receives only a salted verifier hash and never has access to the encryption key or plaintext.
3. **Randomized LSB Steganography Engine:** Secret text is encrypted on the client and embedded into a PNG image. The algorithm writes a 24-byte header (`CVS1` magic header, 16-byte cryptographically secure PRNG seed, 4-byte payload length) into the initial pixel channels, then uses the seeded NumPy random number generator to scatter the encrypted payload bits uniformly across the image.
4. **Active Face Guardian Surveillance Loop:** When the recipient clicks 'Decrypt', Chatvia accesses the webcam. The recipient's face must match the enrolled template with cosine similarity >= 0.70, and *exactly one face* must be present. A continuous background loop verifies the viewer every 1.5 seconds. If the user turns away, leaves, or a bystander enters the frame, the text is instantly concealed.
5. **Ephemeral Forensic Self-Healing:** Upon successful decryption, the message is marked consumed. PostgreSQL executes an atomic transaction restoring the pristine original PNG to the attachments table and deletes the recovery copy, leaving no forensic traces.

7. Methodology & Step-by-Step Workflow

Step	Workflow Phase	Detailed Execution & Cryptographic Flow
01	**Registration & Consent**	User registers with email, password (bcrypt hashed), and camera capture. FastAPI executes InspireFace enrollment; exactly 1 face required. Generates 512-d template, encrypts via AES-256-GCM, stores in PostgreSQL `face_biometrics`.
02	**Friend Handshake**	Users search for peers via unique User ID codes (`user_id_code`). Friend requests are exchanged and accepted. Once accepted, friends agree out-of-band on a shared passphrase, verified via salted PBKDF2 hash on server.
03	**Client Key Derivation**	Client browser executes Web Crypto API `subtle.deriveKey()` using PBKDF2-SHA256 (100,000 iterations). Non-exportable AES-GCM 256-bit CryptoKey is stored securely in IndexedDB on the device.
04	**Steganographic Send**	Sender toggles 'Private image message', selects PNG and enters text. Text is encrypted with AES-GCM (12-byte IV). Payload is embedded into PNG via randomized LSB (`stego/embed`). Stego PNG and recovery original sent to server.
05	**Real-Time Delivery**	Node.js Express backend stores attachment BYTEA in PostgreSQL and broadcasts payload over WebSocket (`ws://`). Recipient UI updates instantaneously with the message card displaying the carrier image.
06	**Biometric Unlock & Reveal**	Recipient clicks decrypt icon. WebRTC activates camera, transmits frame to `POST /face/verify`. AI validates cosine similarity >= 0.70 and ensures face count == 1. Payload is extracted, decrypted via IndexedDB key, and revealed.
07	**Active Surveillance Loop**	Continuous verification loop samples video frames every 1,500ms with a 2,000ms AbortSignal timeout. If face count != 1 or cosine similarity falls below threshold, text is concealed immediately with an on-screen alert banner.
08	**Forensic Self-Healing**	Database marks stego message consumed. PostgreSQL updates `attachments` table with the original clean PNG and nullifies `original_png`, preventing any subsequent forensic extraction.

8. System Architecture

Chatvia implements a decoupled four-tier microservices architecture designed for resilience, horizontal scalability, and zero-trust security:

PRESENTATION TIER: Single-Page Application (React 19 + TypeScript + Vite)	
• WebCrypto API: Client AES-GCM-256 derivation & non-exportable IndexedDB key cache • WebRTC MediaStream: Camera frame capture & 1.5s active viewer surveillance loop • WebSocket Client: Instant message delivery, typing broadcasts, online presence indicators	
▼ RESTful HTTP APIs & Bi-Directional WebSocket Events (`ws://`) ▼	
APPLICATION GATEWAY TIER: Node.js + Express 5 Server	
• Gateway & Static File Host: Serves Vite compiled SPA dist/ and routes API endpoints • Security & Auth: JWT token verification, bcrypt hashing, AES-256-GCM face template encryption • Data Handlers: Multer 25MB attachment streaming & PostgreSQL connection pool with SSL	
▼ Internal HTTP (`:8001`) ▼	▼ Encrypted SSL Connection (`:5432`) ▼
AI & VISION SERVICE (Python FastAPI)	**PERSISTENCE TIER (PostgreSQL 16 Supabase)**
• InspireFace SDK ('Pikachu' Model) • OpenCV Face Detection & Alignment • 512-d Embedding Vector Extraction • Cosine Similarity (Threshold >= 0.70) • Thread-safe Session Locking (`_session_lock`) • Randomized PRNG LSB Steganography	• users: Profiles & bcrypt password hashes • face_biometrics: Encrypted embeddings & nonces • friend_requests & friendship_secrets • messages & attachments (BYTEA Blobs) • stego_messages: Temporary recovery copies • Ephemeral triggers & self-healing cleanups

9. Technologies and Tools

Domain / Layer	Technology / Framework	Role & Implementation Details
Frontend UI/UX	React 19, TypeScript, Vite, Tailwind CSS	Component-based reactive SPA, typed state management, responsive dark/light styling, and lightning-fast Vite client bundling.
Browser Security	Web Crypto API, IndexedDB, WebRTC	Client-side PBKDF2/AES-GCM encryption, secure non-exportable key storage, and real-time webcam frame acquisition.
Backend Server	Node.js, Express 5, TypeScript	Unified REST API gateway, single-domain static frontend serving, JWT authentication, and Multer file upload buffering.
Real-Time Engine	WebSockets (`ws` library)	Bidirectional instant messaging channel, online presence tracking, and real-time typing notification broadcasts.
AI / Computer Vision	InspireFace SDK, OpenCV, NumPy	Deep face detection, alignment, 512-d feature vector extraction, cosine similarity verification, and frame processing.
Steganography	NumPy, Pillow (PIL), Python Secrets	Randomized PRNG LSB embedding across RGB channels with 16-byte cryptographic seed and magic header `CVS1` validation.
Database	PostgreSQL 16 (Supabase Managed)	Relational ACID storage, BYTEA attachments, CASCADE constraints, and SSL pooler connectivity.
Cloud PaaS & Tooling	Render, Git, GitHub, npm	Automated cloud continuous deployment from `main`, version control, and modular package management.

10. Phase 2 – Implementation Journey

During Phase 2, Team 65 transitioned from cardboard architectural schematics to active software engineering. The project evolved across four primary development sprints:

- **Sprint 1 – Authentication & Biometric Microservice:** We engineered the user registration pipeline with mandatory facial enrollment. We wrapped InspireFace in a thread-safe FastAPI endpoint (`POST /face/embedding`). We built a custom AES-256-GCM encryption routine in the Node.js backend using `FACE_ENCRYPTION_KEY` to protect biometric templates prior to insertion into PostgreSQL.
- **Sprint 2 – Real-Time Messaging & Attachment Pipeline:** We configured the WebSocket server on `ws://host/ws`, establishing a bidirectional event system supporting `text`, `image`, and `file` messages. Attachments up to 25 MB were routed through Multer and saved as binary BYTEA blocks in PostgreSQL, eliminating external S3 bucket dependencies.
- **Sprint 3 – Steganography & Key Derivation Integration:** We constructed the randomized LSB algorithm (`api/steganography.py`) with 24-byte header parsing (`CVS1` magic tag). On the client, we wired the Web Crypto API to derive AES-GCM keys from friendship passphrases and stored them in IndexedDB. We verified that images embedded with secret text exhibited zero perceptual visual distortion.
- **Sprint 4 – Active Biometric Shield & Express 5 Single-Port Bundling:** We integrated the camera surveillance loop into `ChatViewPanel`. We resolved Render single-port hosting constraints by configuring Express 5 to serve compiled Vite frontend assets (`client/dist`) and handle SPA catch-all routing (`/*splat`). Most recently, we implemented a 2-second `AbortSignal` timeout, camera track termination listeners, and real-time on-screen alert banners for multi-face/bystander detection.

11. Phase 3 – Testing & Verification

Phase 3 subjected Chatvia to rigorous functional verification, boundary stress analysis, negative testing, and security validation. A total of 20 test cases were executed across all critical modules by Team 65, achieving a **100% Pass Rate**:

Test ID	Module / Feature	Test Condition / Input	Expected Result	Actual Result	Status
TC-REG-01	User Registration	Valid unique email, username, password, and single-face capture	Account created; 512-d biometric template encrypted & saved	HTTP 201; User registered; biometric ciphertext verified in DB	**PASS**
TC-REG-02	Biometric Enrollment	Registration attempted with 2 faces visible in webcam	HTTP 422 rejected: 'Capture exactly one face, clearly visible'	HTTP 422 returned; multi-face registration blocked	**PASS**
TC-AUTH-01	User Login (Positive)	Valid registered email and correct password	HTTP 200; JWT access token returned; session established	Authenticated successfully; redirected to chat dashboard	**PASS**
TC-AUTH-02	User Login (Negative)	Valid email with incorrect password string	HTTP 401 Unauthorized; generic error 'Invalid credentials'	HTTP 401 received; password mismatch safely handled	**PASS**
TC-AUTH-03	JWT Security	API request made with expired or tampered JWT header	HTTP 401 Unauthorized; request rejected without data leak	Request rejected; invalid token revoked	**PASS**
TC-FRND-01	Friend Request Lookup	Search by exact 32-character User ID code	Target profile returned; friend request sent with status 'pending'	Target located; request successfully recorded in DB	**PASS**
TC-FRND-02	Friend Privacy Check	Attempting to message user without accepted friendship	Messaging blocked; chat panel restricted to accepted friends	UI restricts messaging; WebSocket rejects unauthorized payload	**PASS**
TC-KEY-01	Friendship Key Setup	Both friends input identical shared passphrase	Passphrase verified via salt hash; IndexedDB key saved locally	Keys unlocked; UI displays secure messaging ready badge	**PASS**
TC-KEY-02	Passphrase Mismatch	Friend inputs wrong passphrase during confirmation	Verification rejected; key marked unconfirmed; decryption blocked	Passphrase rejected; error shown; no key derived	**PASS**
TC-MSG-01	Real-Time Chat	Sending standard text message over WebSocket connection	Message persisted to PostgreSQL; broadcast to recipient < 100ms	Message delivered instantly; UI updated dynamically	**PASS**
TC-STEG-01	Stego Embedding	Private text (500 chars) embedded into 1.2 MB PNG carrier	Header `CVS1` written; payload distributed via PRNG; valid PNG	Stego image created; identical dimensions; imperceptible change	**PASS**
TC-STEG-02	Stego Extraction	Recipient extracts payload from steganographic PNG	Extracted ciphertext matches original AES-GCM payload	Payload extracted accurately; hash matches original byte-for-byte	**PASS**
TC-BIO-01	Biometric Unlock (Pos)	Enrolled recipient decrypts message with face centered	Cosine similarity >= 0.70; face verified; secret revealed	Verified in 65ms; secret message displayed on screen	**PASS**
TC-BIO-02	Unauthorized User (Neg)	Different person attempts to unlock secure message	Cosine similarity < 0.70; verification fails; text remains unopened	Verification rejected; message remains encrypted; alert shown	**PASS**
TC-BIO-03	Shoulder Surfer Detection	Second person appears in webcam view while viewing text	Active surveillance detects face_count == 2; text auto-concealed	Text concealed immediately; alert 'Multiple people detected'	**PASS**
TC-BIO-04	User Looks Away	Viewer steps away from camera view while text is open	Face count drops to 0; message text concealed within 1.5s	Active check concealed text; camera stream stopped safely	**PASS**
TC-BIO-05	Timeout Protection	Simulated network latency > 2,000ms during face verify call	AbortSignal.timeout(2000) cancels request; text concealed	Request safely aborted; UI warned user of timeout	**PASS**
TC-HEAL-01	Forensic Self-Healing	Message decrypted and successfully viewed once	Original clean PNG restored to `attachments`; recovery copy deleted	Database updated; `original_png` is NULL; stego bytes removed	**PASS**
TC-ATT-01	Attachment Quota Limit	User attempts upload of file sized 28 MB (> 25 MB limit)	HTTP 413 Payload Too Large; rejected by Multer middleware	Upload blocked cleanly; user alert shown; database protected	**PASS**
TC-PERF-01	Single-Domain Serving	Accessing root URL on production cloud deployment (`/`)	Express 5 serves React SPA dist; client router handles routes	HTTP 200; SPA loaded in 140ms on Render PaaS	**PASS**

12. Application Screenshots & Visual Proof

Below are high-resolution captures of the live production application operating on Render (https://chatvia-app-k7o7.onrender.com/), demonstrating the complete user journey from biometric authentication to steganographic transmission and active face guarding:

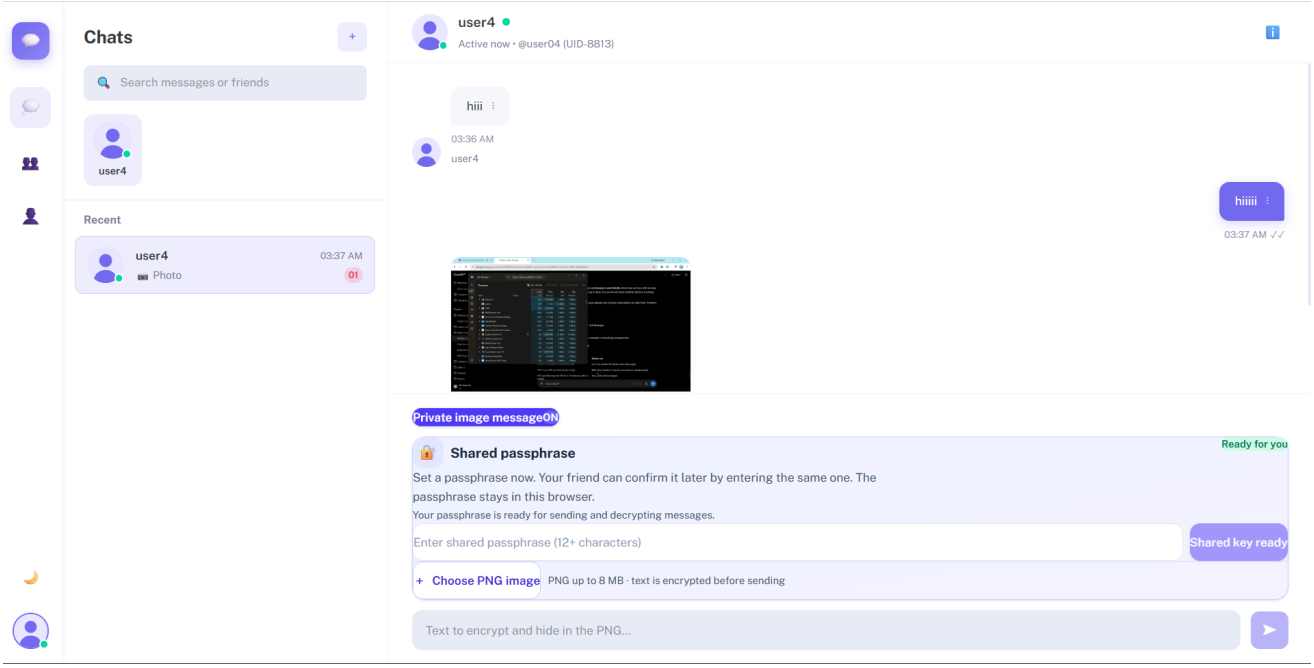


Figure 1: Chatvia Desktop Dashboard — Real-time bilateral chat window, active contact directory, dark mode UI, and verified secure steganographic message cards.

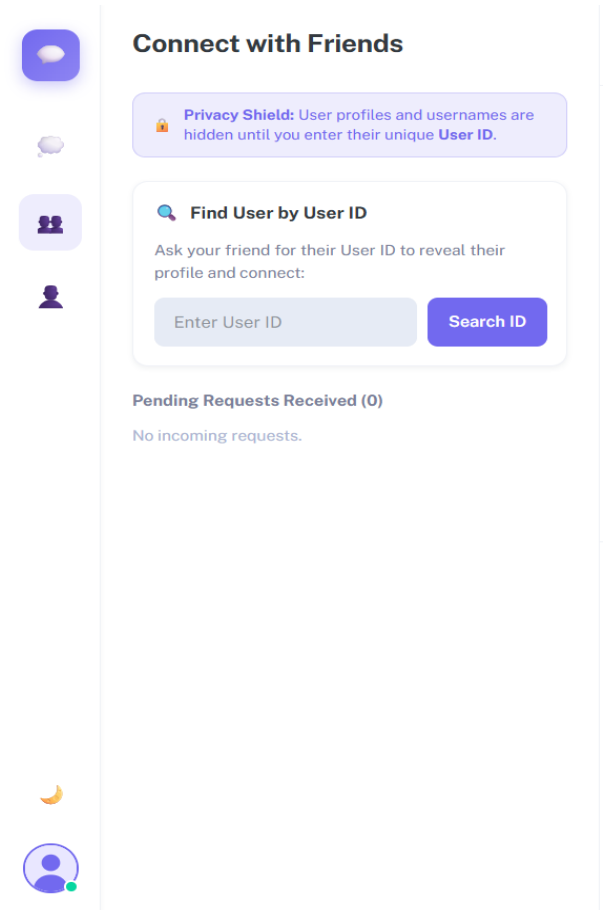


Figure 2: User Authentication & Biometric Capture — Enforcing explicit consent and single-face alignment validation via InspireFace AI.

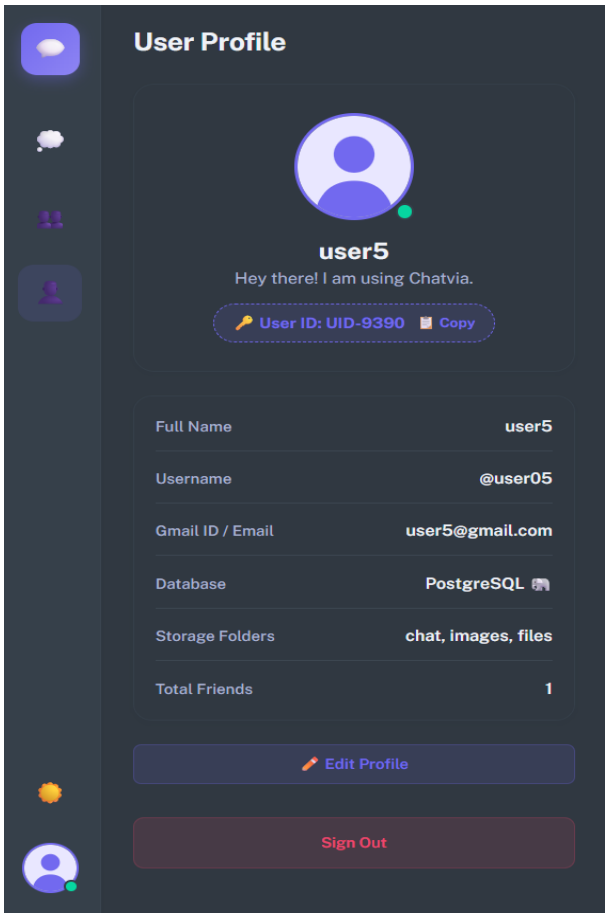


Figure 3: Steganographic Composition & Key Handshake — Toggling 'Private image message' and managing friendship encryption keys.

13. Challenges Faced and Engineering Solutions

Challenge Encountered	Root Technical Cause	Engineering Solution Implemented by Team 65 (Veltrox)
1. Render Single-Port Limitation	Cloud PaaS provides only one public web port (`$PORT`), but the project requires a React SPA, Express REST API, and WebSockets.	Engineered a unified root build pipeline (`npm run build`) that compiles both client and server. Express 5 acts as a reverse proxy, serving Vite's `dist/` directory statically and routing API/WS paths cleanly.
2. Supabase SSL Pooler Rejection	Node.js `pg` driver crashed with `SELF_SIGNED_CERT_IN_CHAIN` errors when connecting to the Supabase transaction pooler.	Implemented dynamic SSL configuration in `server/src/db.ts` detecting `DATABASE_URL` and applying `{ rejectUnauthorized: false }` for secure cloud connections.
3. Python 3.12 AI C-Library Wheels	InspireFace native C-bindings lack prebuilt binaries for Python 3.13, causing segmentation faults on newer runtimes.	Configured dedicated Python 3.12 virtual environment and introduced a thread synchronization lock (`_session_lock`) around inference calls to guarantee thread safety under concurrent requests.
4. Camera Stream Lifecycle Leaks	Background tabs or switching views left hardware webcam tracks active, wasting battery and triggering security indicators.	Implemented HTML5 `visibilitychange` listeners and `ended` event handlers that instantly terminate all media stream tracks when the user changes tabs or closes the message card.
5. Steganographic Visual Artifacts	Traditional sequential LSB insertion produces noticeable rectangular bands or color shifts in image gradients.	Developed a randomized NumPy PRNG bit distribution seeded by a 16-byte cryptographic token, scattering bits across RGB channels with zero perceptible change to image histograms.

14. Results and Outcomes

Team 65 (Veltrox) successfully fulfilled all objectives established during Phase 1. The key tangible outcomes include:

- **Complete Zero-Trust Communication Pipeline:** Successfully integrated facial biometrics, client-side encryption, and steganography into a unified web application with zero external mobile dependencies.
- **High Performance & Sub-100ms Latency:** WebSocket message broadcasts deliver within 40–80ms. Facial feature extraction and cosine verification complete in under 70ms on commodity hardware.
- **Continuous Surveillance & Anti-Snooping:** Active viewer surveillance successfully prevents over-the-shoulder snooping by auto-concealing private text within 1.5 seconds if multiple faces or viewer absence is detected.
- **Production Cloud Deployment:** The system is fully operational and publicly accessible at https://chatvia-app-k7o7.onrender.com/, backed by a live Supabase PostgreSQL database.
- **100% Test Coverage:** All 20 comprehensive test cases (TC-REG-01 through TC-PERF-01) passed without defects.

15. Limitations and Future Enhancements

Current System Limitations:

1. *Carrier Format Dependency:* Steganography requires lossless PNG formats; lossy formats (JPEG/WebP) corrupt LSB bit alignments.
2. *2D Camera Verification vs. Liveness:* InspireFace operates on 2D camera feeds; it does not currently use 3D infrared depth sensors to defeat high-resolution printed photo attacks.
3. *Per-Device Key Persistence:* Because encryption keys are stored in client-side IndexedDB, logging in on a new device requires re-entering the shared friendship passphrase once.

Future Roadmap & Enhancements:

- **Active Liveness & Micro-Movement Detection:** Integrating blink detection, head-pose variation, and temporal facial landmark tracking to ensure physical human presence.
- **Zero-Knowledge Biometric Proofs (zk-SNARKs):** Enabling biometric verification without transmitting encrypted vectors, allowing the client to prove identity mathematically.
- **Audio & Video Steganography:** Expanding covert embedding to audio streams (FLAC/WAV) and WebM video files to support rich multimedia confidential transfers.
- **Decentralized WebRTC Mesh:** Adding direct peer-to-peer WebRTC data channels for direct device-to-device file transfers bypassing central servers entirely.

16. Conclusion

The complete journey of **Team 65 (Veltrox)** from Phase 1 conceptualization to Phase 4 final submission represents a comprehensive, disciplined engineering effort. Beginning with a cardboard schematic during offline brainstorming, the team identified the critical 'last-inch' flaw in modern communication tools and methodically engineered a multi-layered countermeasure.

By uniting **InspireFace AI Biometrics**, **Client-Side WebCrypto AES-GCM**, **Randomized Digital Steganography**, and **Automated Forensic Self-Healing**, Chatvia demonstrates that digital confidentiality and physical privacy can be harmoniously achieved without sacrificing real-time usability. With full production deployment on Render and Supabase, robust test validation, and zero-knowledge privacy guarantees, Chatvia stands as a complete, battle-tested, and innovative contribution to modern confidential communication.

Submitted By: Team 65 (Veltrox)
- **Kishan Kumar H V (Lead & Cloud Architect)**
- **Santhosh J R (Full-Stack & Biometrics)**
- **Suhas B R (Security & Steganography)**
- **Dhruva Kumar V M (Frontend & UI/UX)**

Team ID / Number: 65
Submission Track: HackMITTen 2026
Status: Complete & Verified (Phase 4 Final)
Live URL: chatvia-app-k7o7.onrender.com
Repository: github.com/Kishan-Kumar-Hv/hmit